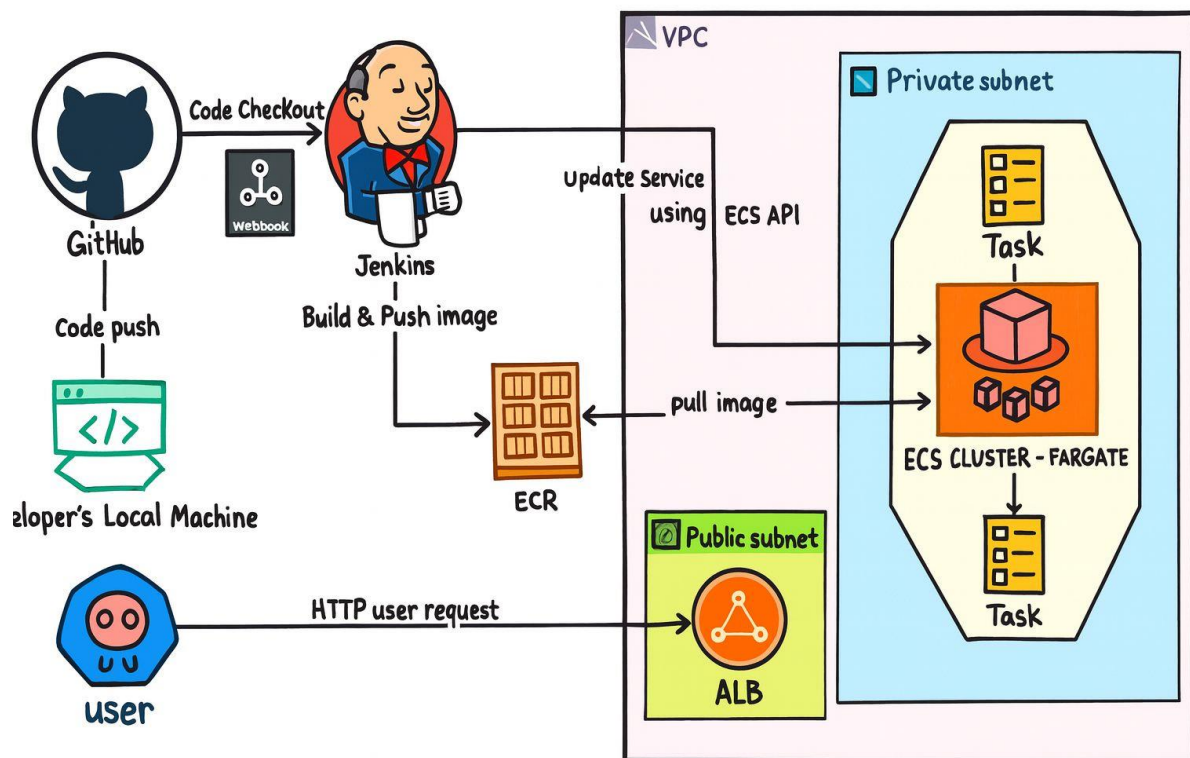


CONTAINERIZE AND DEPLOY FLASK APPLICATIONS

ARCHITECTURE DIAGRAM: Deploy Flask App on AWS Fargate with Jenkins CI/CD



Objective:

Deploy a containerized Flask application on AWS ECS Fargate, expose it using Application Load Balancer (ALB), and automate the deployment process using Jenkins CI/CD pipeline.

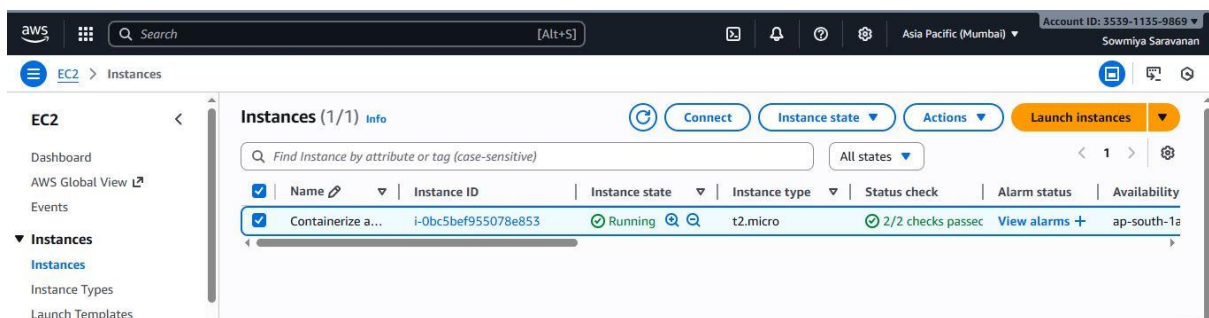
Key Components Involved:

Component	Purpose
Flask App	Sample Python web application to deploy.
Docker	Build and containerize the application.
Amazon ECR	Store Docker images securely in a private registry.
ECS (Fargate)	Serverless container hosting—runs containers without managing servers.
ALB (Application Load Balancer)	Load balancing and public access to the ECS service.
IAM Roles	Provide secure access permissions for ECS, ECR, Jenkins, and other services.

Component	Purpose
Jenkins	CI/CD tool to build Docker images, push to ECR, and deploy to ECS.
GitHub Webhook	Automatically triggers Jenkins when code is pushed to the GitHub repo.

PRE-REQUISITES:

➔ EC2 Instance - Create a EC2 instance (Ubuntu)



➔ Install Docker – Run a command to install docker, “apt install docker.io” “docker –version”

```
root@ip-172-31-38-97:~# docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
root@ip-172-31-38-97:~# systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-11-20 06:35:06 UTC; 22s ago
     TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
      Main PID: 13657 (dockerd)
         Tasks: 8
        Memory: 23.4M (peak: 90.0M)
           CPU: 420ms
      CGroup: /system.slice/docker.service
              └─13657 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Nov 20 06:35:06 ip-172-31-38-97 dockerd[13657]: time="2025-11-20T06:35:06.016147260Z" level=info msg="Loading containers: start."
Nov 20 06:35:06 ip-172-31-38-97 dockerd[13657]: time="2025-11-20T06:35:06.488706691Z" level=info msg="Loading containers: done."
Nov 20 06:35:06 ip-172-31-38-97 dockerd[13657]: time="2025-11-20T06:35:06.538529816Z" level=info msg="Docker daemon" comm
Nov 20 06:35:06 ip-172-31-38-97 dockerd[13657]: time="2025-11-20T06:35:06.538922630Z" level=info msg="Initializing buildk
Nov 20 06:35:06 ip-172-31-38-97 dockerd[13657]: time="2025-11-20T06:35:06.551688131Z" level=warning msg="CDI setup error
Nov 20 06:35:06 ip-172-31-38-97 dockerd[13657]: time="2025-11-20T06:35:06.551710473Z" level=warning msg="CDI setup error
Nov 20 06:35:06 ip-172-31-38-97 dockerd[13657]: time="2025-11-20T06:35:06.579250277Z" level=info msg="Completed buildkit
Nov 20 06:35:06 ip-172-31-38-97 dockerd[13657]: time="2025-11-20T06:35:06.593445331Z" level=info msg="Daemon has complete
Nov 20 06:35:06 ip-172-31-38-97 dockerd[13657]: time="2025-11-20T06:35:06.593572530Z" level=info msg="API listen on /run/
Nov 20 06:35:06 ip-172-31-38-97 systemd[1]: Started docker.service - Docker Application Container Engine.
lines 1-22/22 (END)
```

➔ Install AWS CLI

```
sudo apt update
sudo apt install unzip curl -y
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o
"awscliv2.zip"
```

```
unzip awscli2.zip
sudo ./aws/install
aws --version
```

➔ Install Jenkins

Installation of Java

```
sudo apt update
```

```
sudo apt install fontconfig openjdk-21-jre
```

```
java --version
```

after the installation of java, run this command to install Jenkins

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
```

```
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
```

```
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
```

```
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
```

```
/etc/apt/sources.list.d/jenkins.list > /dev/null
```

```
sudo apt update
```

```
sudo apt install Jenkins
```

```
Jenkins --version
```

After the Jenkins installation, we should check the Jenkins status to run this command like

```
systemctl status jenkins
```

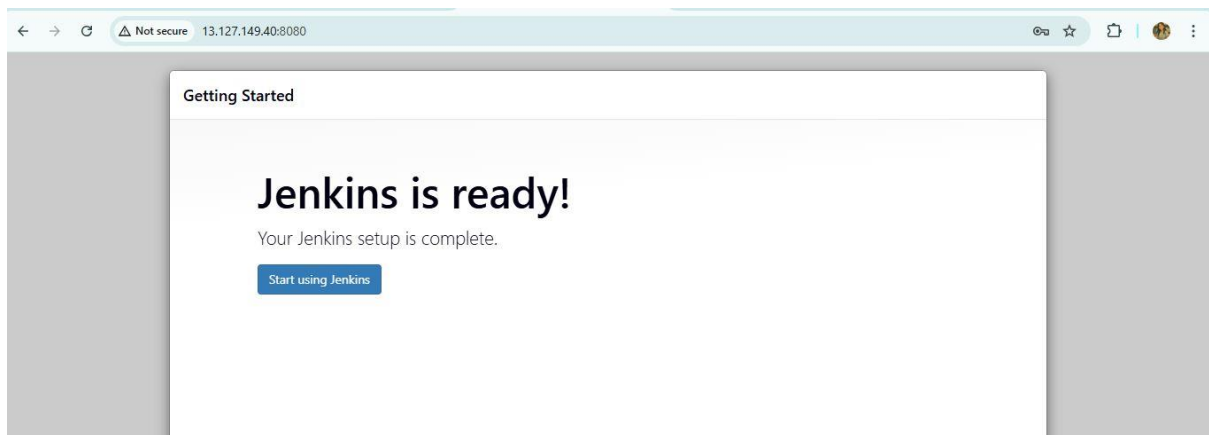
```

root@ip-172-31-38-97:~# systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-11-20 06:28:38 UTC; 3h 10min ago
     Main PID: 12831 (java)
        Tasks: 37 (limit: 1121)
      Memory: 461.4M (peak: 519.2M)
         CPU: 2min 15.215s
        CGroup: /system.slice/jenkins.service
                └─12831 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins>
Nov 20 09:34:20 ip-172-31-38-97 jenkins[12831]: 2025-11-20 09:34:20.761+0000 [id=21] WARNING hudson.securit>
Nov 20 09:34:20 ip-172-31-38-97 jenkins[12831]: 2025-11-20 09:34:20.761+0000 [id=21] WARNING hudson.securit>
Nov 20 09:34:25 ip-172-31-38-97 jenkins[12831]: 2025-11-20 09:34:25.759+0000 [id=19] WARNING hudson.securit>
Nov 20 09:34:25 ip-172-31-38-97 jenkins[12831]: 2025-11-20 09:34:25.762+0000 [id=19] WARNING hudson.securit>
Nov 20 09:34:25 ip-172-31-38-97 jenkins[12831]: 2025-11-20 09:34:25.762+0000 [id=18] WARNING hudson.securit>
Nov 20 09:34:25 ip-172-31-38-97 jenkins[12831]: 2025-11-20 09:34:25.763+0000 [id=18] WARNING hudson.securit>
Nov 20 09:34:30 ip-172-31-38-97 jenkins[12831]: 2025-11-20 09:34:30.760+0000 [id=14] WARNING hudson.securit>
Nov 20 09:34:30 ip-172-31-38-97 jenkins[12831]: 2025-11-20 09:34:30.760+0000 [id=14] WARNING hudson.securit>
Nov 20 09:34:30 ip-172-31-38-97 jenkins[12831]: 2025-11-20 09:34:30.761+0000 [id=16] WARNING hudson.securit>
Nov 20 09:34:30 ip-172-31-38-97 jenkins[12831]: 2025-11-20 09:34:30.763+0000 [id=16] WARNING hudson.securit>
lines 1-20/20 (END)

```

Then, open a browser to hit the pub ip:8080

<http://13.127.149.40:8080>



Finally, give the permission to Jenkins users access the Docker

➔ **Add Jenkins user to the Docker group**

`sudo usermod -aG docker jenkins`

This allows the **jenkins** user to run Docker commands.
Without this, Jenkins cannot do docker build, docker push, etc.

➔ **Change the Docker socket file ownership**

`sudo chown root:docker /var/run/docker.sock`

This makes the Docker engine socket belong to the **docker** group.
Since Jenkins is now part of this group, it can communicate with Docker.

➔ Set proper permissions for the Docker socket

sudo chmod 660 /var/run/docker.sock

This gives **read & write** permission only to:

- root user
- docker group

This is required so Jenkins can use Docker safely.

➔ Restart Jenkins service

sudo systemctl restart jenkins

This applies the new group permissions to the Jenkins process.
(Without restart, Jenkins won't get the new access.)

➔ Switch to Jenkins user and test

sudo su - jenkins

This switches your terminal to act as the **jenkins user**,
so you can test if Docker commands work.

```
root@ip-172-31-38-97:~# sudo chown root:docker /var/run/docker.sock
sudo chmod 660 /var/run/docker.sock
root@ip-172-31-38-97:~# sudo systemctl restart jenkins
root@ip-172-31-38-97:~# sudo su - jenkins
jenkins@ip-172-31-38-97:~$ docker ps
jenkins@ip-172-31-38-97:~$ systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-11-20 15:59:47 UTC; 28s ago
     Main PID: 1742 (java)
       Tasks: 44 (limit: 1121)
    Memory: 353.3M (peak: 353.5M)
       CPU: 15.629s
    CGroup: /system.slice/jenkins.service
            └─1742 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/
```

STEP 1: Create a Sample Application

We first need a small application so we can containerize and deploy it.

Let's take a very simple Python Flask app.

File: app.py

from flask import Flask

```
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from Fargate!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)
```

File: requirements.txt

Flask
gunicorn

STEP 2: Containerize the App (Create a Docker Image)

To run this app inside a container, we need a **Dockerfile**.

File: Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 8080
CMD ["gunicorn", "--bind", "0.0.0.0:8080", "app:app"]
```

Build the Docker Image

docker build -t my-flask-app .

```
Step 5/7 : COPY app.py .
---> 455fceb4a0b5
Step 6/7 : EXPOSE 8080
---> Running in 9b488d62970f
---> Removed intermediate container 9b488d62970f
---> 67822da288b7
Step 7/7 : CMD ["gunicorn", "--bind", "0.0.0.0:8080", "app:app"]
---> Running in 7775926696e9
---> Removed intermediate container 7775926696e9
---> 701b16b95930
Successfully built 701b16b95930
Successfully tagged my-flask-app:latest
root@ip-172-31-38-97:~#
```


Run Locally (Optional Test)

`docker run -itd --name flask-container -p 8090:8080 my-flask-app`

```
root@ip-172-31-38-97:~# docker run -itd --name flask-container -p 8090:8080 my-flask-app
1ef12c50bd25b0af756c8538c991db0ff6f7457fafd7b7db9688033dd476f963
root@ip-172-31-38-97:~# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
1ef12c50bd25	my-flask-app	"gunicorn --bind 0.0..."	About a minute ago	Up About a minute	0.0.0.0:8090->8080/tcp, [::]:8090->8080/tcp

```
flask-container
root@ip-172-31-38-97:~#
```

Open browser:

`http://localhost:8090`

You should see: **"Hello from Fargate!"**



STEP 3: Push Image to Amazon ECR

3.1 Create an ECR Repository

In AWS Console → ECR → Repositories → Create repository

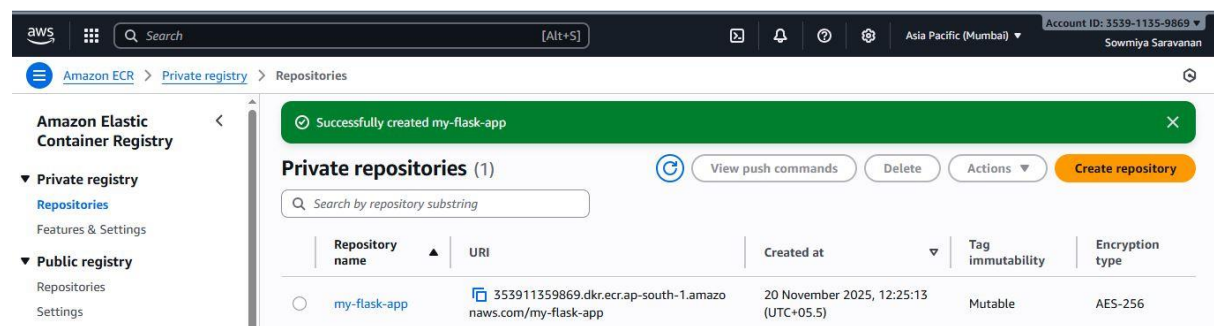
Name: my-flask-app

Visibility: Private

Leave defaults and create.

(or)

`aws ecr create-repository --repository-name my-flask-app --region ap-south-1`



3.2 Login to ECR

```
aws ecr get-login-password --region ap-south-1 \
| docker login --username AWS --password-stdin 353911359869.dkr.ecr.ap-south-1.amazonaws.com
```

```
root@ip-172-31-38-97:~# aws ecr get-login-password --region ap-south-1 \
| docker login --username AWS --password-stdin 353911359869.dkr.ecr.ap-south-1.amazonaws.com

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

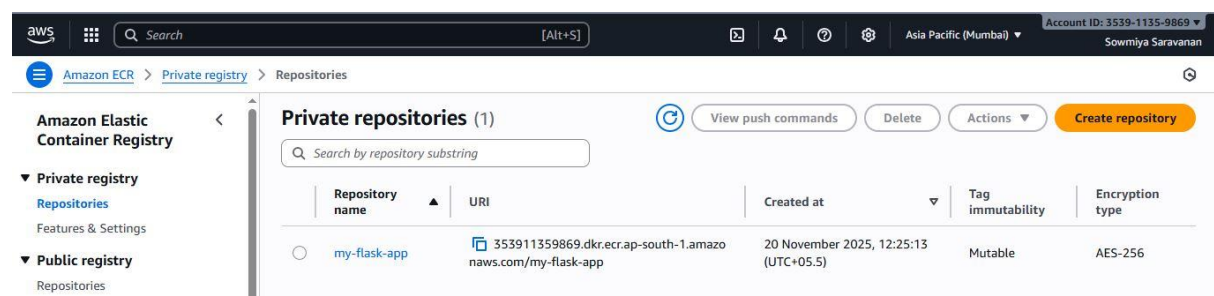
Login Succeeded
root@ip-172-31-38-97:~#
```

3.3 Tag & Push

```
docker tag my-flask-app:latest 353911359869.dkr.ecr.ap-south-1.amazonaws.com/my-flask-app
docker push 353911359869.dkr.ecr.ap-south-1.amazonaws.com/my-flask-app
```

```
root@ip-172-31-38-97:~# docker tag my-flask-app:latest 353911359869.dkr.ecr.ap-south-1.amazonaws.com/my-flask-app
root@ip-172-31-38-97:~# docker push 353911359869.dkr.ecr.ap-south-1.amazonaws.com/my-flask-app
Using default tag: latest
The push refers to repository [353911359869.dkr.ecr.ap-south-1.amazonaws.com/my-flask-app]
4bd9b9706dbd: Pushed
e7f409d3f13b: Pushed
ee8bc2e19952: Pushed
fa19565c2234: Pushed
720ee7aae3ad: Pushed
655ff69eb9c8: Pushed
5c988eaa8862: Pushed
70a290c5e58b: Pushed
latest: digest: sha256:8c68c3fbf3e72dae2673b3ab5a20bf313aff9c4700b45fd5cec37d02e15bfd56 size: 1990
root@ip-172-31-38-97:~# docker images
REPOSITORY                                TAG      IMAGE ID       CREATED        SIZE
353911359869.dkr.ecr.ap-south-1.amazo    latest   701b16b95930   31 minutes ago 141MB
my-flask-app                             latest   701b16b95930   31 minutes ago 141MB
python                                    3.11-slim 040af88f5bce   2 days ago    124MB
root@ip-172-31-38-97:~#
```

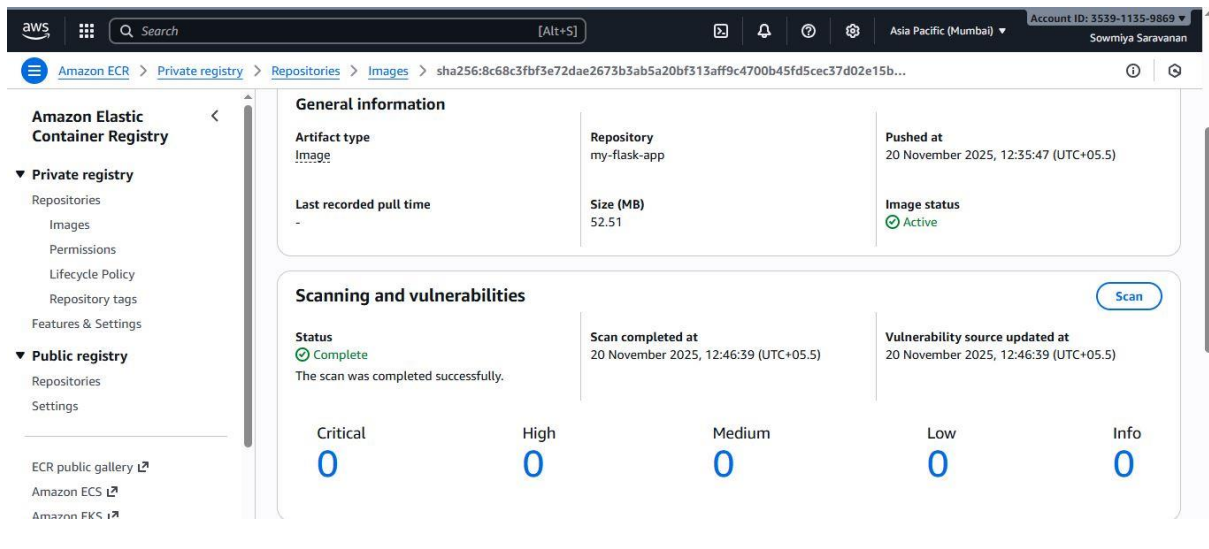
Now your container image is stored in ECR.



The screenshot shows the AWS Management Console interface for the Amazon Elastic Container Registry (ECR). The left sidebar shows the navigation menu with 'Amazon ECR' selected. The main content area is titled 'Private repositories (1)' and contains a table with the following data:

Repository name	URI	Created at	Tag immutability	Encryption type
my-flask-app	353911359869.dkr.ecr.ap-south-1.amazonaws.com/my-flask-app	20 November 2025, 12:25:13 (UTC+05.5)	Mutable	AES-256

They show there is no vulnerability in our image.



✓ STEP 4: Deploy Using AWS Fargate (ECS)

AWS Fargate runs containers **without servers**. You don't need EC2.

To run our container, we need:

- ✓ ECS Cluster
- ✓ Task Definition
- ✓ Service
- ✓ Load Balancer (ALB)
- ✓ Security Groups

Let's keep this very simple.

4.1 Create ECS Cluster

In ECS Console:

Clusters → Create cluster

Choose: Networking only (Fargate) (or new console: “Create cluster” → choose Fargate-compatible)

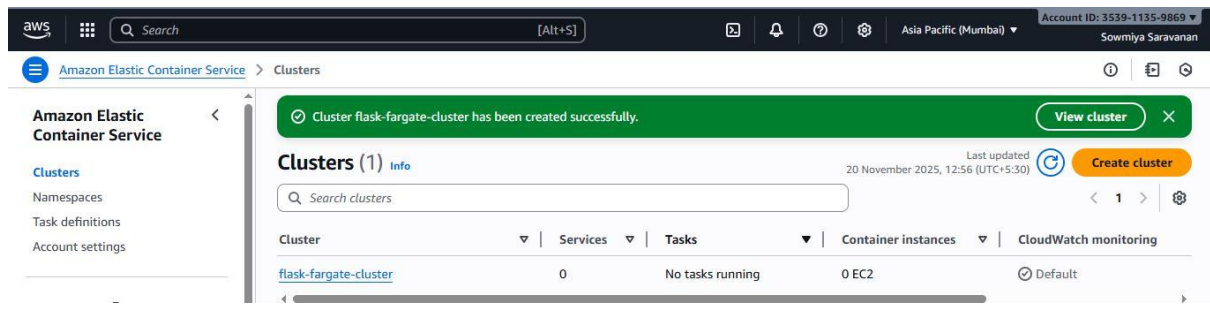
Name: flask-fargate-cluster

Select your VPC & subnets (private).

Create.

(or)

aws ecs create-cluster --cluster-name flask-fargate-cluster



4.2 Create an Execution Role for ECS Task

Task definition defines about container specification.

Task Role (optional - unless the running container needs to access AWS services)

Task Execution Role (This role allows ECS to pull images from ECR)

Create Task execution role in IAM: `ecsTaskExecutionRole`

Attach policy: `AmazonECSTaskExecutionRolePolicy` (AWS Managed)

(or)

This role allows ECS to pull images from ECR.

Commands simplified:

aws iam create-role --role-name ecsTaskExecutionRole --assume-role-policy-document file://trust-policy.json

aws iam attach-role-policy --role-name ecsTaskExecutionRole --policy-arn arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy

In ECS console:

Task definition → Create new task definition

Launch type: Fargate

Name: flask-fargate-task

Task memory: 0.5 GB (or 512 MiB)

Task CPU: 0.25 vCPU (or 256)

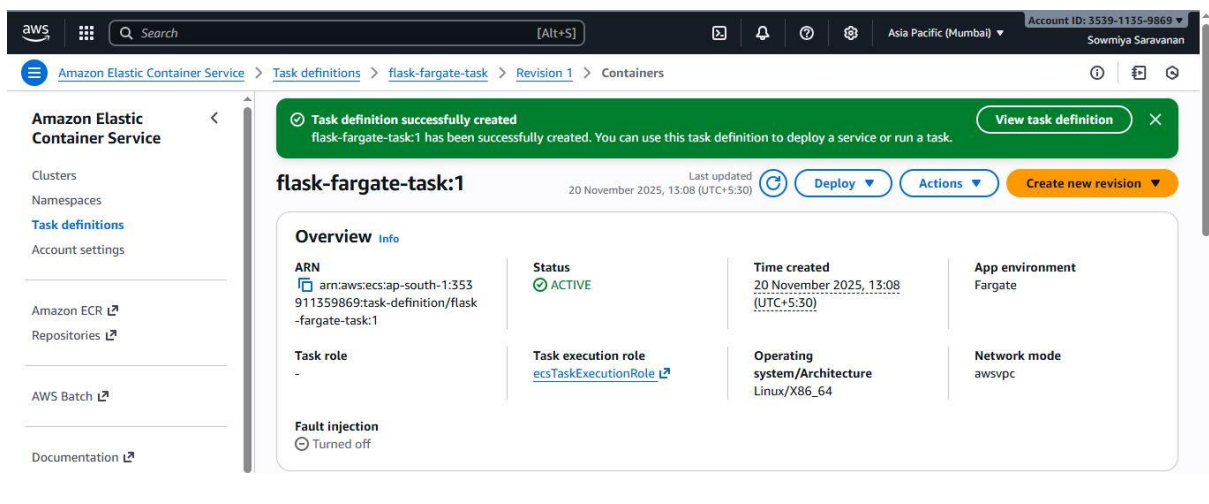
Add container:

Container name: flask-app

Image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/flask-fargate-demo:latest

Port mappings: container port 8080 / TCP

Save & create.



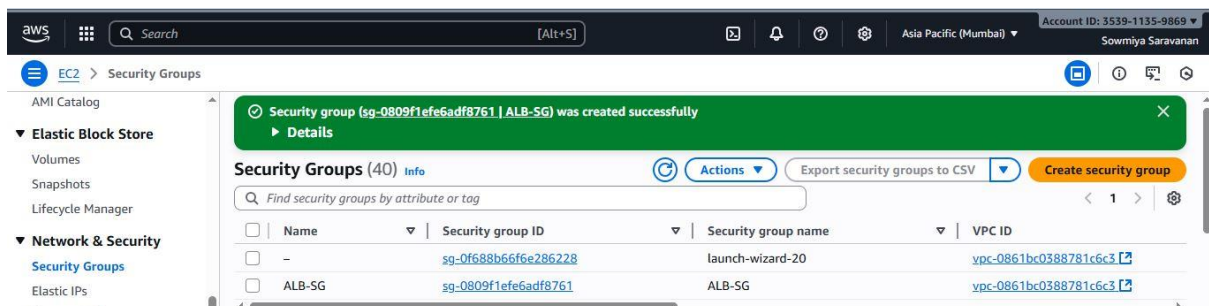
4.3 Create Security Groups

First, we can create a security group for ALB

ALB SG - "ALB-SG"

Inbound: 80 from 0.0.0.0/0

Outbound: all

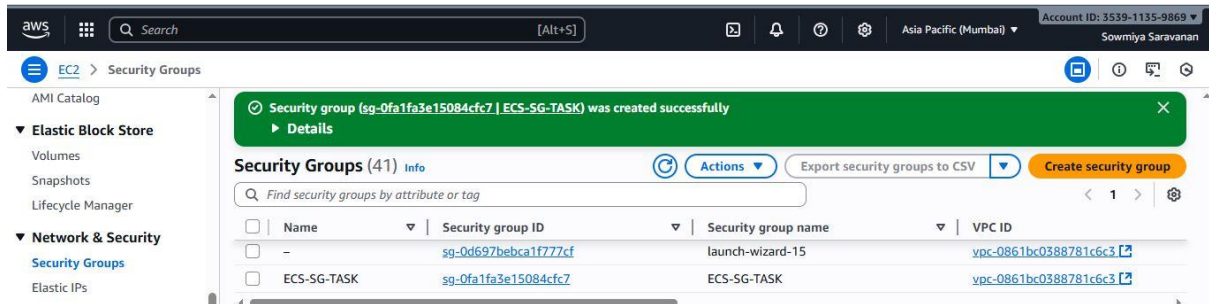


then, we can create a security group for ECS-SG-TASK

ECS Tasks SG - “ECS-SG-TASK”

Inbound: 8080 from ALB-SG

Outbound: 0.0.0.0/0 (for ECR, logs, etc.)



4.4 Create Application Load Balancer (ALB)

ALB sits in front (public subnet) and sends traffic to ECS FARGATE

- Accepts traffic from Internet (port 80)
- Sends traffic to tasks running in private subnets
- Performs health checks

In EC2 → Load Balancers → Create load balancer:

Type: Application Load Balancer

Name: FLASK-ALB

Scheme: Internet-facing

IP type: IPv4

Select 2+ public subnets

Security Group: “ALB-SG” with:

Inbound: HTTP 80 (0.0.0.0/0) (and/or HTTPS 443 if you add cert later)

Target Group

Target type: IP

Name: FLASK-TG

Protocol: HTTP

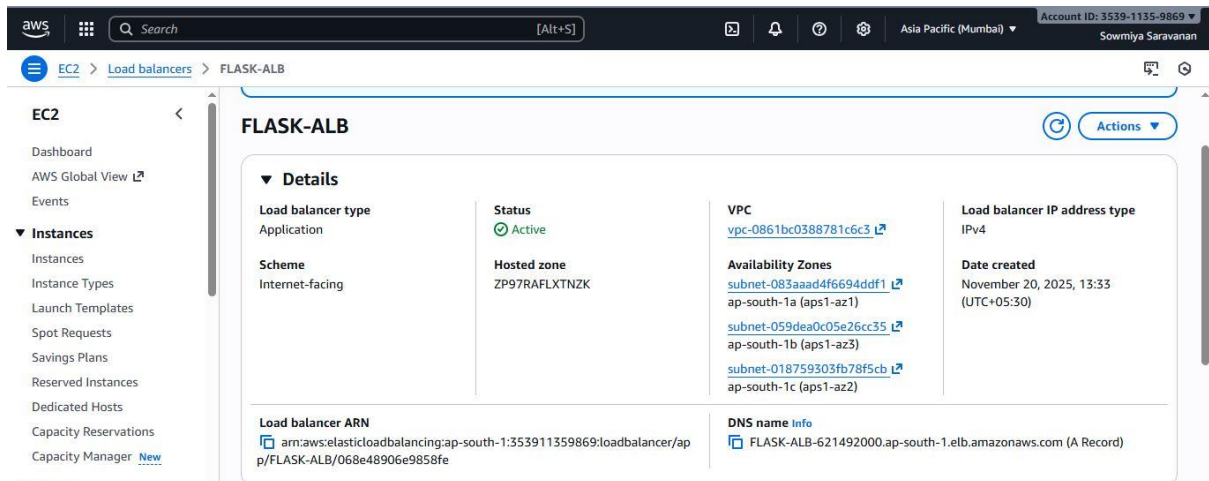
Port: 8080

Health check path: /

Health check protocol: HTTP

Listener:

Port 80 → forward to FLASK-TG



4.5 Create ECS Service (Fargate + ALB)

Service manages your task on Fargate

- Connects tasks to ALB (Load Balancing)
- Does Auto-restart, Auto-scale and Rolling updates

In ECS → Cluster → flask-fargate-cluster → Create service

Launch type: Fargate

Task definition: flask-fargate-task:1

Service name: flask-fargate-service

Desired tasks: 2 (for HA)

Deployment type: Rolling

VPC: your main VPC

Subnets: private subnets (recommended) or public for simple demo

Security group: ECS-SG-TASK

Load balancing:

Select Application Load Balancer

Choose your ALB: FLASK-ALB

Listener: HTTP 80

Target group: select FLASK-TG

Health check path /

Create service.

(or)

`aws ecs create-service \`

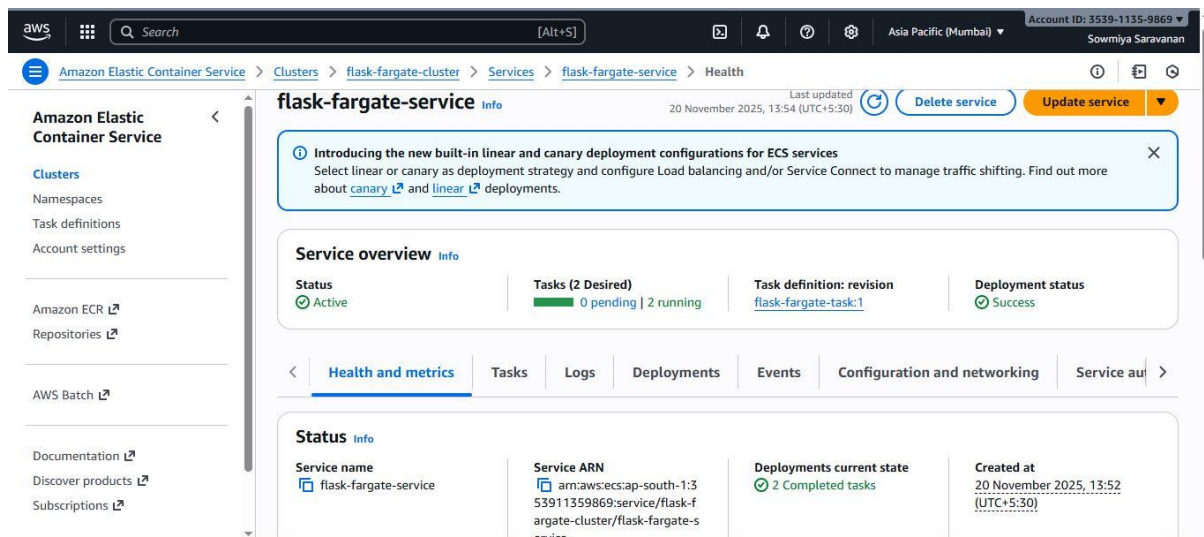
`--cluster flask-fargate-cluster \`

`--service-name flask-fargate-service \`

`--task-definition flask-fargate-task \`

`--desired-count 2 \`

`--launch-type FARGATE`



After tasks go RUNNING and targets become healthy, hit the ALB DNS name in browser:

Open the ALB DNS URL in browser:

[http:// flask-alb-621492000.ap-south-1.elb.amazonaws.com/](http://flask-alb-621492000.ap-south-1.elb.amazonaws.com/)

You should see:

"Hello from Fargate!"



This means:

- Container → running on Fargate
- Fargate → registered with ALB
- ALB → sending traffic
- App → responding

DONE! 🚀

STEP 5: CI/CD Using Jenkins

When you push code → Jenkins should automatically:

1. Build Docker image
2. Push to ECR
3. Update ECS task definition
4. Redeploy ECS service

Exactly same as GitHub Actions... but now in **JENKINS STYLE**.

Jenkins Pipeline (Jenkinsfile)

```
pipeline {  
    agent any
```

```

environment {
    AWS_REGION    = "ap-south-1"
    AWS_ACCOUNT_ID = "353911359869"
    ECR_REPO_NAME  = "my-flask-app"
    ECR_REGISTRY   =
"$${AWS_ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com"
    IMAGE_TAG      = "v${env.BUILD_NUMBER}"

    TASK_FAMILY    = "flask-fargate-task"
    CLUSTER_NAME   = "flask-fargate-cluster"
    SERVICE_NAME   = "flask-fargate-service"

    // ✓ Correct container name
    CONTAINER_NAME = "flask-container"
}

stages {

    stage('Checkout Code') {
        steps {
            git branch: 'main', url: 'https://github.com/sowmiya6498/flask-
fargate.git'
        }
    }

    stage('Build Docker Image') {
        steps {
            sh """

```

```
    echo "Building Docker Image..."

    docker build -t ${ECR_REPO_NAME}:${IMAGE_TAG} ./flask-
app
    """

    }

}

stage('Login to ECR & Tag Image') {
    steps {
        withCredentials([
            usernamePassword(
                credentialsId: 'aws-creds',
                usernameVariable: 'AWS_ACCESS_KEY_ID',
                passwordVariable: 'AWS_SECRET_ACCESS_KEY'
            )
        ]) {

            sh """

            export AWS_ACCESS_KEY_ID=${AWS_ACCESS_KEY_ID}

            export
AWS_SECRET_ACCESS_KEY=${AWS_SECRET_ACCESS_KEY}

            echo "Logging in to ECR..."

            aws ecr get-login-password --region ${AWS_REGION} \
                | docker login --username AWS --password-stdin
${ECR_REGISTRY}

            echo "Tagging Image..."
```

```
        docker tag ${ECR_REPO_NAME}:${IMAGE_TAG}
        ${ECR_REGISTRY}/${ECR_REPO_NAME}:${IMAGE_TAG}
```

```
        ""
```

```
    }
```

```
}
```

```
}
```

```
stage('Push Image to ECR') {
```

```
    steps {
```

```
        sh ""
```

```
        echo "Pushing image..."
```

```
        docker push
```

```
        ${ECR_REGISTRY}/${ECR_REPO_NAME}:${IMAGE_TAG}
```

```
        ""
```

```
    }
```

```
}
```

```
stage('Register New Task Definition Revision') {
```

```
    steps {
```

```
        withCredentials([
```

```
            usernamePassword(
```

```
                credentialsId: 'aws-creds',
```

```
                usernameVariable: 'AWS_ACCESS_KEY_ID',
```

```
                passwordVariable: 'AWS_SECRET_ACCESS_KEY'
```

```
            )
```

```
        ])
```

```
        sh ""
```

```
export AWS_ACCESS_KEY_ID=${AWS_ACCESS_KEY_ID}

export
AWS_SECRET_ACCESS_KEY=${AWS_SECRET_ACCESS_KEY}

echo "Registering NEW ECS Task Definition..."

NEW_TASK_DEF_ARN=\$(aws ecs register-task-definition \
    --family ${TASK_FAMILY} \
    --execution-role-arn
arn:aws:iam::${AWS_ACCOUNT_ID}:role/ecsTaskExecutionRole \
    --network-mode awsvpc \
    --requires-compatibilities FARGATE \
    --cpu 256 \
    --memory 512 \
    --container-definitions '[
    {
        "name": "${CONTAINER_NAME}",
        "image":
"${ECR_REGISTRY}/${ECR_REPO_NAME}:${IMAGE_TAG}",
        "essential": true,
        "portMappings": [
            {
                "containerPort": 8080,
                "protocol": "tcp"
            }
        ]
    }
    ]'\
```



```

        --query "taskDefinition.taskDefinitionArn" \
        --output text)

    echo "NEW_TASK_DEF_ARN=\$NEW_TASK_DEF_ARN" >
new-task-def.env

    echo "Task Definition Created: \$NEW_TASK_DEF_ARN"
    ""
}
}
}

stage('Deploy New Revision to ECS Service') {
    steps {
        withCredentials([
            usernamePassword(
                credentialsId: 'aws-creds',
                usernameVariable: 'AWS_ACCESS_KEY_ID',
                passwordVariable: 'AWS_SECRET_ACCESS_KEY'
            )
        ]) {

            sh """
            export AWS_ACCESS_KEY_ID=${AWS_ACCESS_KEY_ID}
            export
AWS_SECRET_ACCESS_KEY=${AWS_SECRET_ACCESS_KEY}

            NEW_TASK_DEF_ARN=\$(cat new-task-def.env | cut -d '=' -
f2)

```

```
echo "Deploying NEW revision to ECS service..."
```

```
aws ecs update-service \  
    --cluster ${CLUSTER_NAME} \  
    --service ${SERVICE_NAME} \  
    --task-definition \${NEW_TASK_DEF_ARN}
```

```
echo "ECS Service Updated Successfully!"
```

```
"""
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

Jenkins Flow:

Step 1 → Checkout Code

Jenkins pulls your latest project from GitHub.

Step 2 → Login to Amazon ECR

Jenkins logs into ECR using AWS credentials.

Step 3 → Build Docker Image

Jenkins builds a new image using your Dockerfile.

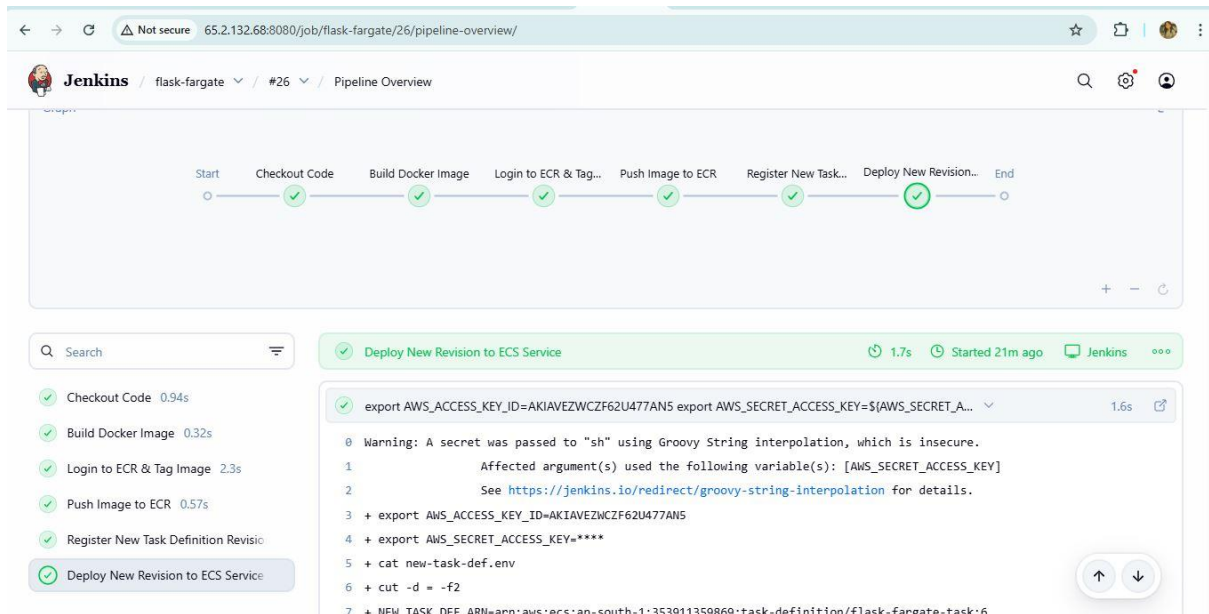
Step 4 → Tag & Push Image

The image is pushed to ECR with a unique tag (commit ID).

Step 5 → Deploy to ECS Fargate

Jenkins forces ECS to start a new deployment → Containers restart with new image.

FINAL OUTCOME



- ✓ You build a full DevOps pipeline
- ✓ Jenkins automatically builds & deploys
- ✓ ECR stores your images securely
- ✓ ECS Fargate runs containers serverlessly
- ✓ ALB exposes your app to the internet
- ✓ Every code push = New version automatically deployed

STEP 6: Configure Webhook (GitHub → Jenkins Trigger)

Goal: When you push code to GitHub, Jenkins should automatically start the pipeline.

6.1 Enable Webhook Trigger in Jenkins

1. Open Jenkins → your job (flask-fargate).
2. Click **Configure**.
3. Go to **Build Triggers**.
4. Tick **GitHub hook trigger for GITScm polling**.
5. Save.

This tells Jenkins:

→ “Whenever GitHub sends a webhook, run this job.”

6.2 Create Webhook in GitHub

1. Open your GitHub repo:
2. <https://github.com/sowmiya6498/flask-fargate>
3. Go to:
Settings → **Webhooks** → **Add Webhook**
4. Fill the fields:

Field	Value
-------	-------

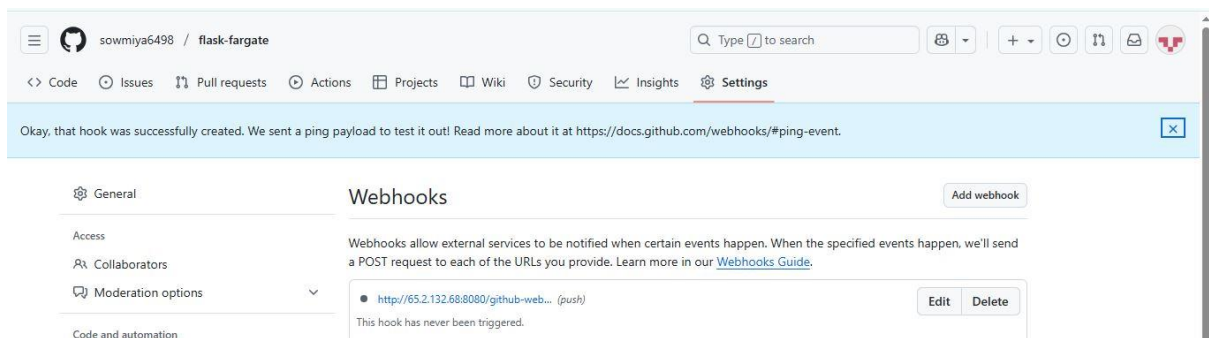
Payload URL	<code>http://65.2.132.68:8080/github-webhook/</code>
--------------------	--

Content Type	<code>application/json</code>
---------------------	-------------------------------

Secret	(keep empty for now OR add secret)
---------------	------------------------------------

Events	Choose Just the push event
---------------	-----------------------------------

4. Click **Add Webhook**.



6.3 After Webhook Setup

Whenever a developer update or changes the code, the code pushed to gitrepo and the webhook trigger the build automatically.

```
git add .  
git commit -m "my update"  
git push
```

```

root@ip-172-31-38-97:~/project# git add .
git commit -m "my update"
git push
[main 17b47ec] my update
Committer: root <root@ip-172-31-38-97.ap-south-1.compute.internal>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

1 file changed, 1 insertion(+), 1 deletion(-)
Username for 'https://github.com': sowmiya6498
Password for 'https://sowmiya6498@github.com':
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 427 bytes | 427.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/sowmiya6498/flask-fargate.git
   0ed5ce9..17b47ec  main -> main
root@ip-172-31-38-97:~/project# █

```

Immediately GitHub will send a webhook to Jenkins:

- GitHub says: “New code pushed!”
- Jenkins receives the webhook
- Jenkins automatically starts the pipeline
 - ✓ Checkout Code
 - ✓ Build Docker Image
 - ✓ Push to ECR
 - ✓ Create new Task Definition
 - ✓ Deploy to ECS Fargate

You do NOT need to click “Build Now” anymore.

The build triggers automatically within **1–3 seconds** after the push.

How to confirm it works

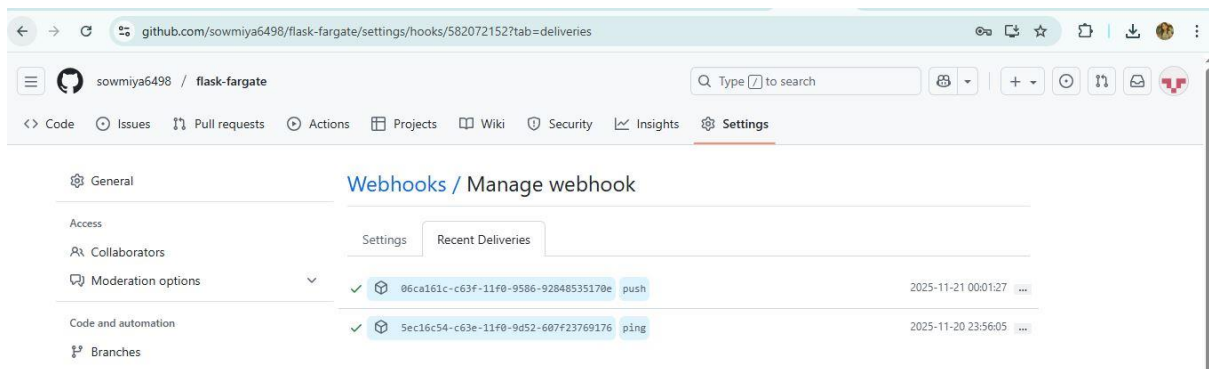
After you push code:

Step 1 — Check GitHub

Go to:

Repo → Settings → Webhooks → Recent Deliveries

You should see:

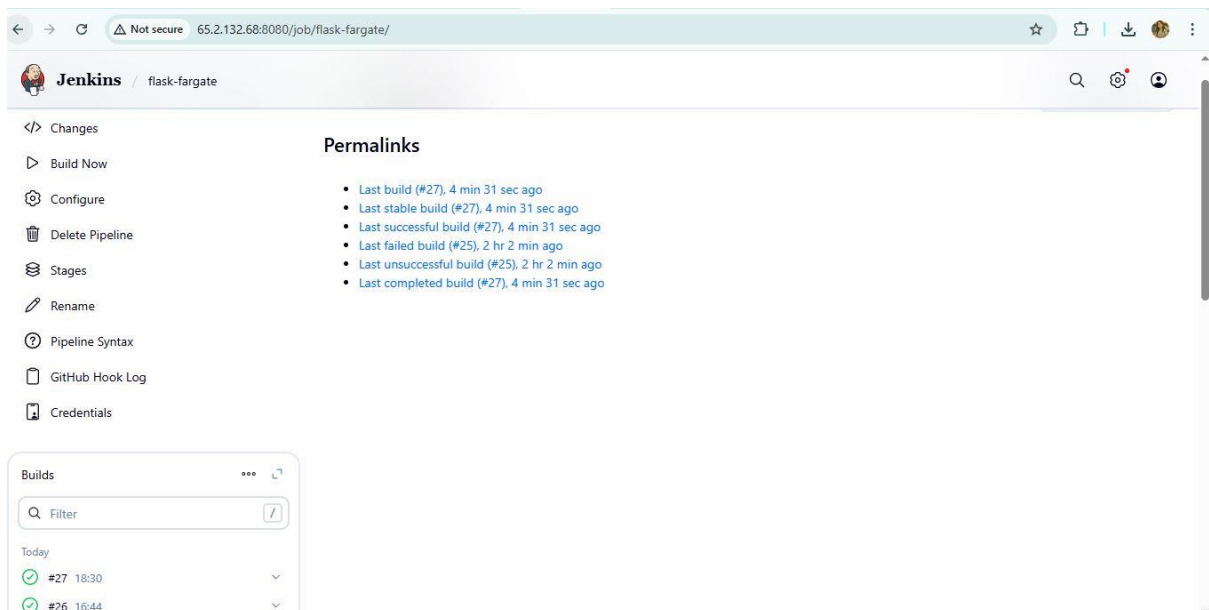


Step 2 — Check Jenkins

Go to your pipeline:

<http://65.2.132.68:8080/job/flask-fargate/>

You should see a new build starting automatically.



Step 3 — Open a browser to hit ALB URL:

<http://flask-alb-621492000.ap-south-1.elb.amazonaws.com/>

You saw the new updated code



→ This confirms your new container image is live on Fargate.

Final Result

Your CI/CD pipeline is automatically deploying new code to AWS Fargate!

- No manual build
- No manual deployment
- Everything is automated

Your setup is now a **production-grade CI/CD pipeline**.

🌐 Traffic Flow

Internet → Application Load Balancer → ECS Fargate → Container → APP
